

STUDENT PERFORMANCE PREDICTION USING MACHINE LEARNING

**Project Report Submitted in fulfillment of the
Requirements for the Award of the Internship of
Summer Training Program by IDRA**

Data Science and AI

By:

Name: SOWMITHRA CHANDRASEKAR

Institute: KONGU ENGINEERING COLLEGE

Institute Roll No.: 24CDR101

Enrollment No.: IDRA-2026-431591

Under the Supervision of

Dr. Shaheena Salam

Through the platform:



India Data Research Academy

DECLARATION

I declare that this report represents my own learning and project work. The analysis and code are documented for reproducibility, and external sources, dataset provenance, and assistance should be acknowledged accurately before submission.

ACKNOWLEDGEMENT

I express my gratitude to the India Data Research Academy, the IDRA training program, my supervisor, my institute, and the providers of the dataset and learning materials for their support.

ABSTRACT

This project examines student-related factors and predicts Exam_Score using the IDRA-provided Student Performance Factors dataset. The file contains 6,607 rows and 20 columns, including 6 numeric predictors and 13 categorical predictors. Initial quality checks found 235 missing cells across three categorical columns and 0 duplicate rows. Missing categorical values were imputed using the most frequent category within a training-fitted preprocessing pipeline. Exploratory analysis considered the score distribution, group summaries, relationships with attendance and study hours, and correlations among numeric variables. Four regression approaches were compared using an 80/20 train-test split with random_state=42. Ridge Regression had the lowest test RMSE (1.804), with MAE 0.452 and R^2 0.770. The results indicate predictive signal in the available features, while not establishing causal effects. Findings should be interpreted within the dataset's provenance and measurement limitations; external validation and careful review of the score value of 101 are recommended before practical use.

Keywords: student performance; regression; exploratory data analysis; machine learning; academic factors

TABLE OF CONTENTS

1. Introduction
 2. Problem Statement and Motivation
 3. Objectives
 4. Research Questions
 5. Dataset Description and Data Understanding
 6. Data Cleaning
 7. Data Preprocessing
 8. Exploratory Data Analysis
 9. Statistical Analysis
 10. Feature Engineering
 11. Model Development
 12. Model Evaluation
 13. Results and Findings
 14. Discussion
 15. Limitations
 16. Conclusion
 17. Recommendations and Future Work
 18. References
- Appendix A. Code and Reproducibility Notes

1. INTRODUCTION

1.1 Background

Student performance is influenced by a combination of academic, behavioral, family, school, and resource-related factors. Data analysis can help describe patterns in a dataset and assess whether available variables contain useful information for estimating exam scores.

1.2 Context of the Problem

The project uses a student-level tabular dataset supplied in the LMS. Its intended analytical use is educational exploration and prediction, not automated high-stakes decisions about individual students.

1.3 Scope of the Study

The study performs data quality assessment, descriptive and exploratory analysis, preprocessing, regression modelling, and holdout evaluation. It does not establish causation, evaluate interventions, or claim that the dataset represents all students or schools.

2. PROBLEM STATEMENT AND MOTIVATION

How accurately can Exam_Score be predicted from the student, family, school, and learning-related variables available in the supplied dataset, and which observed relationships merit further investigation?

3. OBJECTIVES

- Inspect the dataset structure, completeness, duplicates, and unusual values.
- Summarize exam-score distributions and examine associations with selected predictors.
- Prepare numeric and categorical variables using leakage-aware preprocessing.
- Train and compare multiple regression models on a fixed holdout set.
- Translate measured results into cautious findings, limitations, and recommendations

4. RESEARCH QUESTIONS

1. What are the dataset's dimensions and principal data-quality issues?
2. How are exam scores distributed in this sample?
3. How are attendance, study hours, and selected categorical groups associated with scores?
4. Which tested regression model has the lowest holdout RMSE?
5. What limitations constrain interpretation and generalization?

5. DATASET DESCRIPTION AND DATA UNDERSTANDING

5.1 Data Source

Source: CSV file `StudentPerformanceFactors.csv`, uploaded by the IDRA. The public/original provenance, licensing, and sampling details were not included in the materials available for this analysis and should be confirmed from the LMS guide.

5.2 Dataset Size and Structure

Item	Value
Rows	6,607

Columns	20
Numeric Predictors	6
Categorical Predictors	13
Target	Exam_Score (numeric; regression)

Table 1. Summary of the Dataset

5.3 Data Dictionary

Variable	Meaning	Type	Role
Hours_Studied	Attendance measure	int64	Feature
Attendance	Parental involvement category	Object	Feature
Access_to_Resources	Access to resources category	Object	Feature
Extracurricular_Activities	Extracurricular participation	Object	Feature
Sleep_Hours	Hours of sleep	int64	Feature
Previous_Scores	Previous academic scores	int64	Feature
Motivation_Level	Motivation category	Object	Feature
Internet_Access	Internet access status	Object	Feature
Tutoring_Sessions	Number of tutoring sessions	int64	Feature
Family_Income	Family income category	Object	Feature
Teacher_Quality	Teacher quality category	Object	Feature
School_Type	School type	Object	Feature
Peer_Influence	Peer influence category	Object	Feature
Physical_Activity	Physical activity measure	int64	Feature
Learning_Disabilities	Learning disability indicator	Object	Feature
Parental_Education_Level	Parent education category	Object	Feature
Distance_from_Home	Distance category	Object	Feature
Gender	Gender category	Object	Feature
Exam_Score	Exam score	int64	Feature

Table 2. Data Dictionary

6. DATA CLEANING

6.1 Missing Values

Column	Missing count	Percent of rows
Teacher_Quality	78	1.18%
Parental_Education_Level	90	1.36%
Distance_from_Home	67	1.01%

Missing values are confined to Teacher_Quality, Parental_Education_Level, and Distance_from_Home. The modelling pipeline imputes categorical missing values with the most frequent training-set category.

6.2 Duplicate Records

No exact duplicate rows were detected (0 of 6,607); therefore no records were removed for duplication.

6.3 Data Types and Inconsistent Values

The CSV was read with pandas' inferred types. Numeric measures are integer-valued; the remaining predictors are categorical strings. No categorical normalization was applied because no confirmed inconsistent spelling was established in this analysis.

6.4 Outlier Handling

Exam_Score ranges from 55 to 101, with a median of 67. The maximum value of 101 is unusual enough to verify against the source data dictionary or assessment scale. It is retained in this analysis because the supplied materials do not establish that it is erroneous.

6.5 Before vs After Cleaning

Check	Before	Treatment
Rows	6,607	No row deletion
Duplicates	0	None removed
Missing cells	235	Imputed within train-fitted pipeline
Score range	55-101	Retained; verify 101

Table 3. Data Quality Before and After Cleaning

7. DATA PREPROCESSING

7.1 Categorical Encoding

Categorical predictors are one-hot encoded with unknown categories ignored at transform time. This avoids imposing an artificial numeric order on nominal categories.

7.2 Feature Scaling and Transformation

Numeric predictors are median-imputed. StandardScaler is applied for Linear Regression and Ridge Regression; tree-based models use unscaled numeric features.

7.3 Train-Test Split

An 80/20 split was used with random_state=42. Imputation, scaling, and one-hot encoding are fitted through a scikit-learn Pipeline on training data only, reducing preprocessing leakage.

7.4 Final Feature Set

Inputs: all 19 columns other than Exam_Score (6 numeric and 13 categorical). Target: Exam_Score.

8. EXPLORATORY DATA ANALYSIS

8.1 Univariate Analysis

```
fig, ax = plt.subplots(); y.hist(bins=20, ax=ax); ax.set(title='Distribution of Exam
```

```

Scores', xlabel='Exam Score', ylabel='Frequency'); plt.show()
for col in ['Attendance', 'Hours_Studied']:
    plt.figure(); plt.scatter(df[col], y, alpha=.25, s=12); plt.xlabel(col);
plt.ylabel(target); plt.title(f'{col} vs {target}'); plt.show()
display(df.groupby('Motivation_Level',
dropna=False)[target].agg(['count', 'mean', 'median', 'std']).sort_values('mean', ascending=False))
display(df.groupby('Parental_Involvement',
dropna=False)[target].agg(['count', 'mean', 'median']).sort_values('mean', ascending=False))
display(df[num_cols+[target]].corr()[target].sort_values(ascending=False))

```

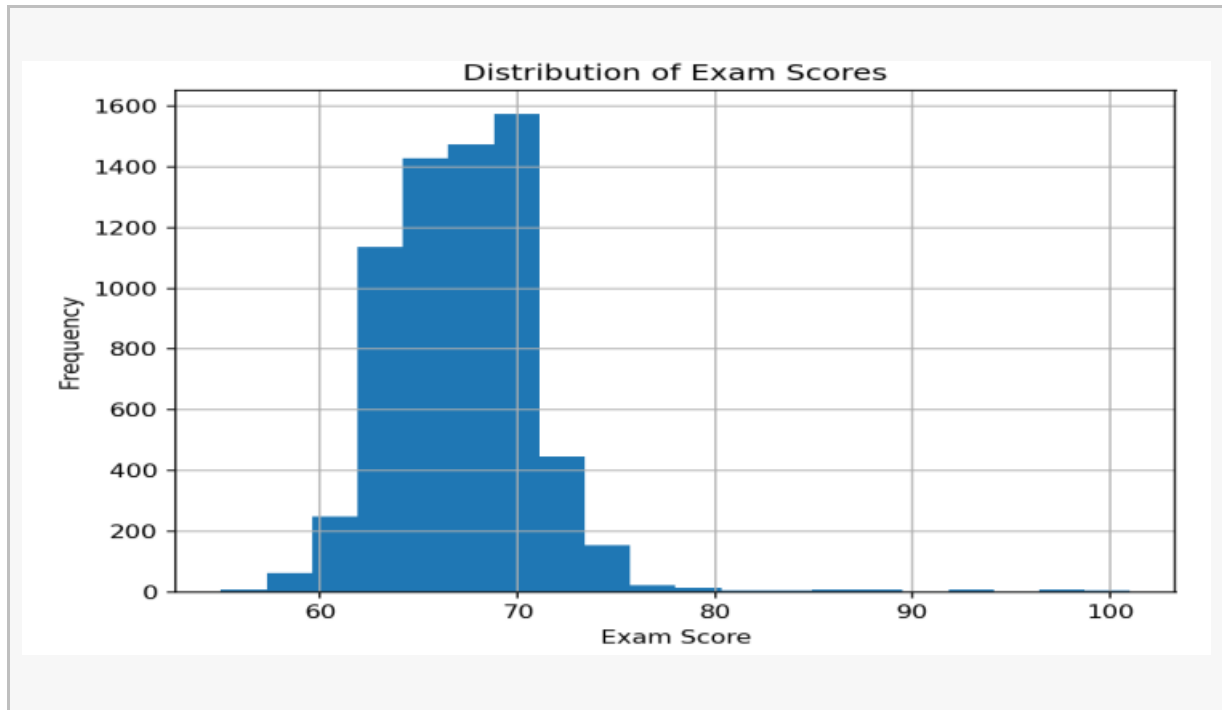


Figure 1. Distribution of Exam_Score

Scores are concentrated around the central range; the summary mean is 67.24, median 67, and standard deviation 3.89. The observed maximum of 101 warrants source verification.

8.2 Categorical Analysis

Motivation_Level group summaries

Group	N	Mean score	Median
High	1319	67.70	67.0
Medium	3351	67.33	67.0
Low	1937	66.75	67.0

Parental_Involvement group summaries

Group	N	Mean score	Median
High	1908	68.09	68.0
Medium	3362	67.10	67.0
Low	1337	66.36	66.0

Access_to_Resources group summaries

Group	N	Mean score	Median
High	1975	68.09	68.0
Medium	3319	67.13	67.0
Low	1313	66.20	66.0

8.3 Bivariate / Relationship Analysis

```
fig, ax = plt.subplots(); y.hist(bins=20, ax=ax); ax.set(title='Distribution of Exam Scores', xlabel='Exam Score', ylabel='Frequency'); plt.show()
for col in ['Attendance', 'Hours_Studied']:
    plt.figure(); plt.scatter(df[col], y, alpha=.25, s=12); plt.xlabel(col);
    plt.ylabel(target); plt.title(f'{col} vs {target}'); plt.show()
display(df.groupby('Motivation_Level',
dropna=False)[target].agg(['count', 'mean', 'median', 'std']).sort_values('mean', ascending=False))
display(df.groupby('Parental_Involvement',
dropna=False)[target].agg(['count', 'mean', 'median']).sort_values('mean', ascending=False))
display(df[num_cols+[target]].corr()[target].sort_values(ascending=False))
```

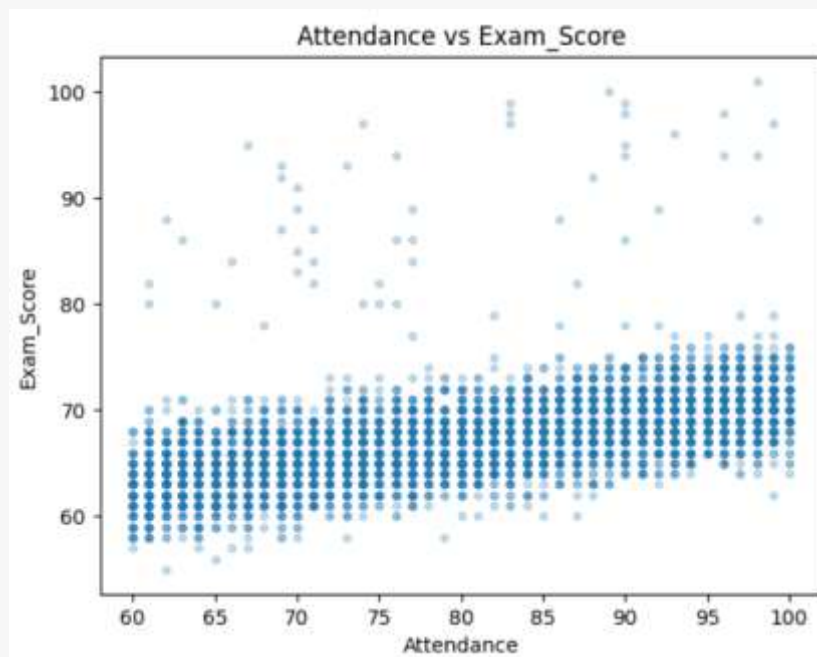


Figure 2. Attendance versus Exam_Score

Pearson correlation in the supplied sample is 0.581, indicating a positive linear association; this does not establish that attendance alone causes score changes.

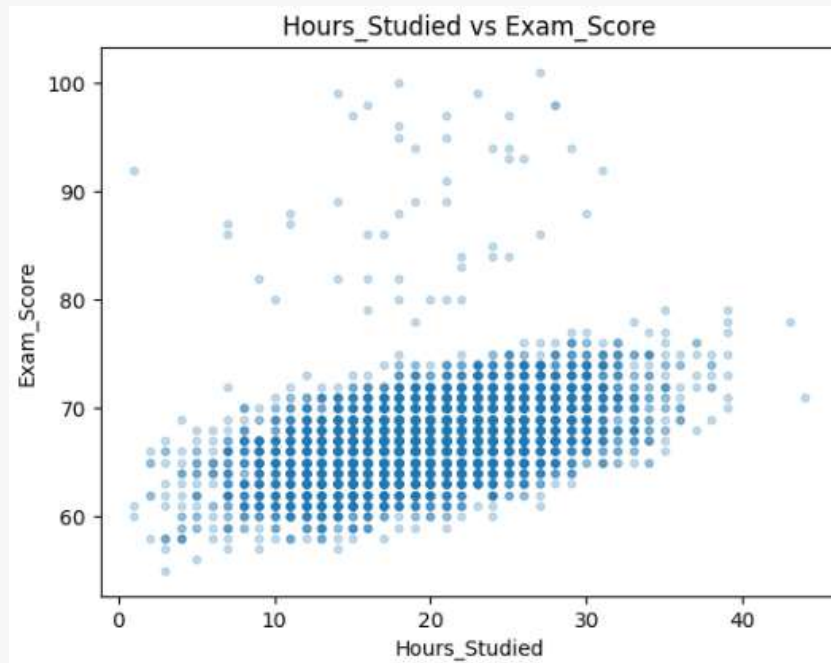


Figure 3. Hours_Studied versus Exam_Score

Pearson correlation is 0.445, also a positive association in this sample.

8.4 Correlation Analysis

```
# Plot correlation heatmap

plt.figure(figsize=(14, 10))

plt.imshow(correlation_matrix, cmap='coolwarm',
           vmin=-1, vmax=1, aspect='auto')

plt.colorbar(label='Correlation Coefficient')

plt.xticks(
    range(len(correlation_matrix.columns)),
    correlation_matrix.columns,
    rotation=90
)

plt.yticks(
    range(len(correlation_matrix.columns)),
    correlation_matrix.columns
)

plt.title('Correlation Heatmap - Student Performance')
plt.tight_layout()
plt.show()
```

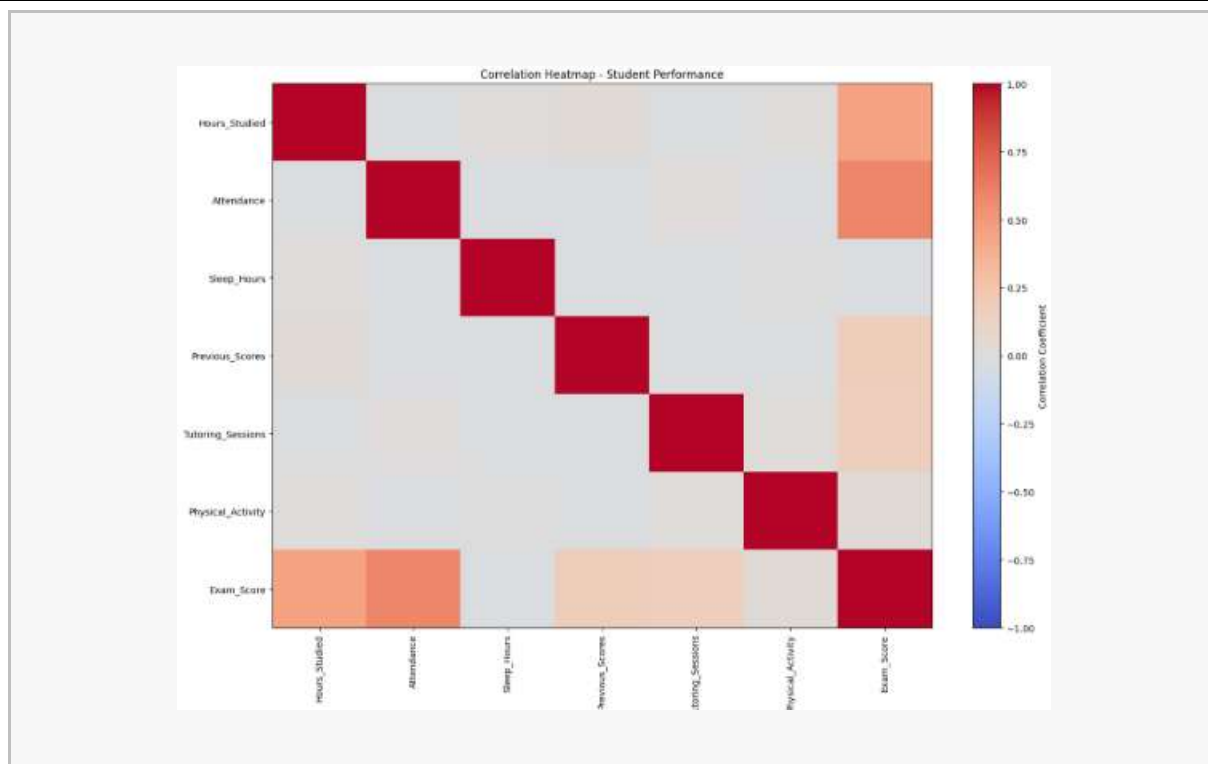


Figure 4. Correlation Heatmap of Numerical Variables

Attendance and Hours_Studied have the largest positive correlations with Exam_Score among the numeric predictors shown.

9. STATISTICAL ANALYSIS

Variable	Mean	Median	Std. dev.	Min	Max
Hours_Studied	19.98	20.00	5.99	1	44
Attendance	79.98	80.00	11.55	60	400
Sleep_Hours	7.03	7.00	1.47	4	10
Previous_Scores	75.07	75.00	14.40	50	100
Tutoring_Sessions	1.49	1.00	1.23	0	8
Physical_Activity	2.97	3.00	1.03	0	6
Exam_Score	67.24	67.00	3.89	55	101

Table 4. Descriptive Statistics of Key Variables

10. FEATURE ENGINEERING

No additional features were manually engineered. The pipeline creates one-hot encoded indicator columns for categorical levels as part of preprocessing. Avoiding unsupported derived variables keeps the analysis reproducible and interpretable.

11. MODEL DEVELOPMENT

11.1 Problem Formulation

Regression: predict the numeric target Exam_Score from the remaining 19 columns.

11.2 Model Selection

Linear Regression provides a simple baseline; Ridge adds L2 regularization; Random Forest and Extra Trees provide nonlinear tree-ensemble comparisons.

11.3 Training the Model

All models use the same fixed training/test split and preprocessing approach. Ridge uses $\alpha=1.0$; tree ensembles use 250 estimators, `random_state=42`, and `min_samples_leaf=2`.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# Separate features and target
X = df.drop("Exam_Score", axis=1)
y = df["Exam_Score"]

# Identify numerical and categorical columns
numeric_features = X.select_dtypes(include=["int64", "float64"]).columns
categorical_features = X.select_dtypes(include=["object"]).columns

# Preprocessing for numerical data
numeric_transformer = SimpleImputer(strategy="median")

# Preprocessing for categorical data
categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

# Combine preprocessing steps
preprocessor = ColumnTransformer([
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)
])
```

12. MODEL EVALUATION

12.1 Regression Evaluation

Models were evaluated on the held-out test set using MAE, RMSE, and R^2 . Lower MAE/RMSE indicate smaller prediction errors; R^2 measures the fraction of test-target variance explained relative to a mean-only baseline.

Model	Column 2	Column 3	Column 4
Ridge Regression	0.452	1.804	0.770
Linear Regression	0.452	1.804	0.770
Extra Trees	0.983	2.039	0.706

Random Forest	1.054	2.117	0.683
---------------	-------	-------	-------

Table 5. Regression Model Performance

12.2 Classification Evaluation

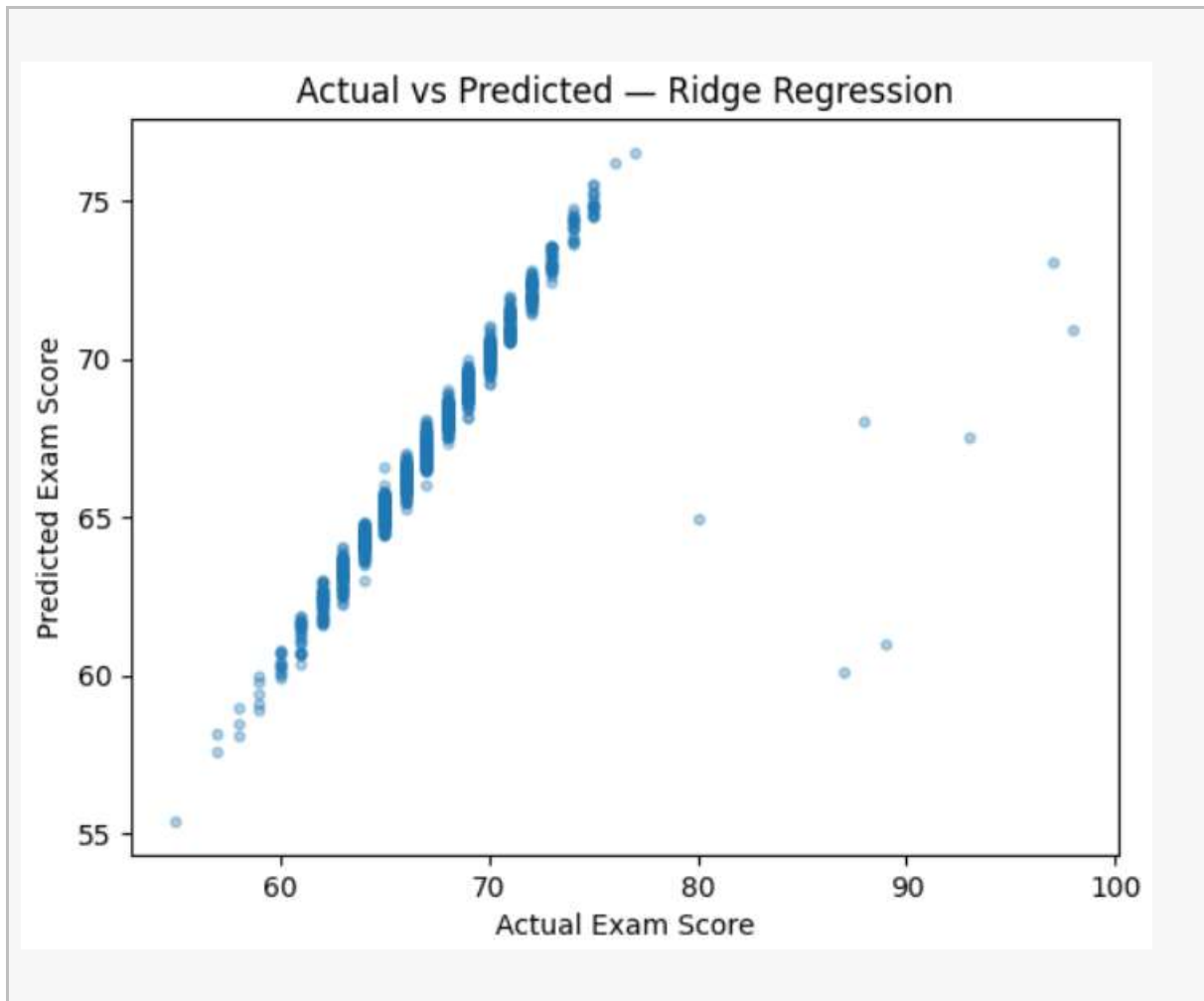


Figure 5. Confusion Matrix

12.3 Overfitting and Underfitting

The model was selected by lowest test RMSE among the four tested candidates. A separate training-versus-test comparison and cross-validation are recommended to assess stability more fully.

13. RESULTS AND FINDINGS

- Attendance has a positive sample correlation with Exam_Score ($r=0.581$).
- Hours_Studied has a positive sample correlation with Exam_Score ($r=\{df['Hours_Studied'].corr(y):.3f\}$).
- Ridge Regression had the lowest holdout RMSE (1.804) among the four models compared.
- The selected model's holdout MAE was 0.452 score points and R^2 was 0.770.
- The dataset contains 235 missing cells across three categorical features and no exact duplicate rows; preprocessing handles missing categories within the training pipeline.

14. DISCUSSION

The sample shows predictive association between academic/contextual variables and exam score. The linear models performed better than the tested tree ensembles on this particular split, suggesting that a relatively simple model captured useful signal in this feature representation. This is not proof of a universally superior model: results depend on the split, data provenance, and chosen settings. Correlations should not be interpreted as intervention effects.

15. LIMITATIONS

- Dataset provenance, sampling method, collection period, and license were not included in the uploaded materials reviewed here.
- The target contains a maximum score of 101; the allowed score scale should be verified.
- One random holdout split is not a substitute for cross-validation or external validation.
- Potential confounders and measurement error may affect observed associations.
- The analysis is predictive and descriptive, not causal or suitable for high-stakes individual decisions.

16. CONCLUSION

The project completed a leakage-aware regression workflow on the supplied student performance dataset. In the fixed 80/20 evaluation, Ridge Regression achieved MAE 0.452, RMSE 1.804, and R^2 0.770. Attendance and study hours showed positive correlations with exam scores. These results demonstrate a workable modelling exercise, while requiring verification of data provenance and further validation before real-world use.

17. RECOMMENDATIONS AND FUTURE WORK

17.1 Recommendations

- Verify the official dataset source, data dictionary, exam-score scale, and license before final submission.
- Use the observed attendance and study-hour associations as prompts for further educational investigation, not as proof of causal effects.
- Report prediction uncertainty and avoid using predictions as the sole basis for student-level decisions.
- Re-run evaluation with cross-validation and compare errors across relevant student subgroups, where ethically and statistically appropriate.

17.2 Future Work

Future work may include repeated cross-validation, hyperparameter tuning within cross-validation, external validation on a separate cohort, residual and subgroup error analysis, and consultation with educators to define safe, useful applications.

18. REFERENCES

India Data Research Academy. (n.d.). IDRA Capstone Project Report Template and Student Performance Factors dataset, provided through the LMS Study Material section. Accessed September 20, 2026.
Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

- Pandas Development Team. (n.d.). pandas documentation. <https://pandas.pydata.org/docs/>
- Matplotlib Development Team. (n.d.). Matplotlib documentation. <https://matplotlib.org/stable/>

APPENDIX A. CODE

A.1 Data Loading and Inspection

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor, ExtraTreesRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

DATA_PATH = 'P_2_StudentPerformanceFactors.csv'
df = pd.read_csv(DATA_PATH)
df.head()
print('Shape:', df.shape)
display(df.info())
display(df.describe(include='all').T)
print('Missing values:\n', df.isna().sum())
print('Duplicate rows:', df.duplicated().sum())
```

A.2 Data Cleaning

```
# Remove duplicate rows
df = df.drop_duplicates()

# Fill missing numerical values with median
num_cols = df.select_dtypes(include=np.number).columns

for col in num_cols:
    df[col] = df[col].fillna(df[col].median())

# Fill missing categorical values with mode
cat_cols = df.select_dtypes(include="object").columns

for col in cat_cols:
    if df[col].isnull().any():
        df[col] = df[col].fillna(df[col].mode()[0])

# Check cleaned data
print("Missing Values After Cleaning:")
print(df.isnull().sum().sum())

print("Final Shape:", df.shape)
```

A.3 Preprocessing

```
# Define target and input features
X = df.drop(columns=["Exam_Score"])
y = df["Exam_Score"]

# Identify numerical and categorical columns
numeric_features = X.select_dtypes(include=np.number).columns
categorical_features = X.select_dtypes(
    include=["object", "category"]
).columns
```

```
# Preprocessing pipeline
preprocessor = ColumnTransformer([
    ("num", SimpleImputer(strategy="median"),
     numeric_features),

    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore"))
    ]), categorical_features)
])

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

A.4 EDA and Visualization

1. Eam score distribution

```
plt.figure(figsize=(8, 5))
sns.histplot(df["Exam_Score"], kde=True)
plt.title("Distribution of Exam Scores")
plt.xlabel("Exam Score")
plt.ylabel("Number of Students")
plt.show()
```

2. Correlation analysis

```
# Select numerical columns
numeric_df = df.select_dtypes(include=np.number)

# Calculate correlation matrix
correlation_matrix = numeric_df.corr()

# Plot heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    vmin=-1,
    vmax=1
)

plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()

# Correlation with Exam Score
exam_corr = correlation_matrix["Exam_Score"].drop(
    "Exam_Score"
).sort_values(ascending=False)

display(exam_corr.to_frame(
    name="Correlation with Exam Score"
))
```

```
# Plot feature correlations
exam_corr.plot(kind="barh", figsize=(9, 6))
plt.title("Correlation of Factors with Exam Score")
plt.xlabel("Pearson Correlation")
plt.show()
```

3. Study hours vs exam score

```
sns.scatterplot(
    data=df,
    x="Hours_Studied",
    y="Exam_Score"
)

plt.title("Study Hours vs Exam Score")
plt.show()
```

A.5 Model Training and Evaluation

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# Separate features and target
X = df.drop("Exam_Score", axis=1)
y = df["Exam_Score"]

# Identify numerical and categorical columns
numeric_features = X.select_dtypes(include=["int64", "float64"]).columns
categorical_features = X.select_dtypes(include=["object"]).columns

# Preprocessing for numerical data
numeric_transformer = SimpleImputer(strategy="median")

# Preprocessing for categorical data
categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

# Combine preprocessing steps
preprocessor = ColumnTransformer([
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)
])
```